Smart Campus Wi-Fi Network Anomaly Detection using Machine Learning (Isolation Forest)

Problem Statement:

A university campus network serves thousands of students through dozens of Wi-Fi access points. Network staff usually learn about a problem only after users complain: a hostel access point that is quietly uploading gigabytes of data, a burst of failed log-in attempts against a lab network, a flood of traffic that slows a whole building, or an access point whose hardware is failing. Fixed alarm limits do not work well because normal load differs a great deal between a library at noon, a hostel at 9 p.m., a canteen at lunch time and an office on a Sunday, so a limit that is sensible for one place is either silent or constantly firing for another.

There is, therefore, a need for a data-driven system that learns what is normal for every access point and hour of the week and raises an alert, with a severity and a probable cause, when the traffic is abnormal. This project designs and implements a Smart Campus Wi-Fi Anomaly Detection system in Python 3 using JupyterLab. It performs data cleaning, behaviour-based feature engineering, and unsupervised anomaly detection with an Isolation Forest tuned on a later validation window, and evaluates the trained model on an unseen, still later period using Precision, Recall, F1 score, ROC-AUC and PR-AUC, a comparison with two baseline detectors, and an estimate of the alert workload (alerts and false alarms per day).

Objectives:

1. To collect and organise an hourly Wi-Fi access-point log containing connected devices, download and upload volume, failed log-ins, latency, packet loss and the zone of each access point.
2. To clean the data by removing duplicate log lines, repairing controller gaps in latency and packet loss, and validating the numeric ranges.
3. To engineer meaningful features (upload/download ratio, megabytes per device and comparisons with the same hour of the previous weeks) from the cleaned data.
4. To design, tune (grid search scored on a later validation window), train and save an Isolation Forest model that is fitted without any incident labels.
5. To evaluate the model on an unseen, chronologically later test period, report Precision, Recall, F1, ROC-AUC and PR-AUC, compare it with baseline detectors, and estimate the alert workload.
6. To assign a severity and a probable cause to every alert and persist it (alert time, access point, score, severity, cause, timestamp) in an SQLite database for record-keeping.

Software / Hardware Required:

• Operating System: Windows / Linux / macOS (executed on Linux)

• Environment: JupyterLab (notebook: notebooks/WiFi_Anomaly_Detection.ipynb)

• Language: Python 3.x (executed with Python 3.13.16)

• Machine Learning: scikit-learn 1.9.1 (IsolationForest, LocalOutlierFactor)

• Data Handling: pandas 3.0.5, NumPy 2.5.3

• Visualization: Matplotlib 3.11.2

• Model Persistence: joblib 1.6.0

• Database: SQLite3 (alert log)

• Dataset: Synthetic hourly Wi-Fi log data – 28,840 records (10 access points × 120 days × 24 hours = 28,800 plus 40 duplicated log lines, 2025-01-01 to 2025-04-30) generated by data/generate_dataset.py with a fixed random seed (42)

• Hardware: Standard CPU-only computer; no GPU required

Data note: no real network logs were available, so the dataset is synthetic and is produced from explicit, documented rules (zone load profiles, per-device traffic, noise, four injected incident types and a few legitimate auditorium events). The incident labels are used only to choose settings on a validation window and to evaluate the result; the model is trained without them. All results in this report therefore describe performance on simulated data. Replacing data/wifi_traffic.csv with real controller logs that have the same columns allows the same code to be retrained without modification.

Project Directory Structure:

The project ("WiFiAnomalyDetection") is organized with separate folders for the dataset, source modules, trained model, notebook, outputs, database and screenshots, as shown below.

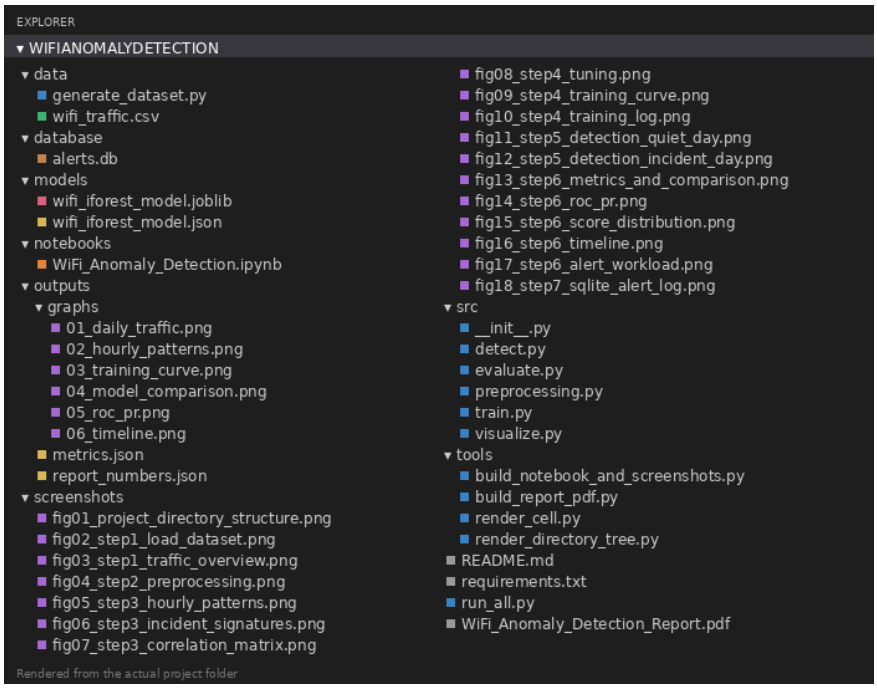


Fig 1: Project directory structure of WiFiAnomalyDetection (generated from the actual project folder)

Procedure:

1. Prepare the dataset: run data/generate_dataset.py to create data/wifi_traffic.csv (hourly logs per access point with devices, traffic, failed log-ins, latency and packet loss).

2. Install the libraries listed in requirements.txt and launch JupyterLab; open notebooks/WiFi_Anomaly_Detection.ipynb.

3. Load the CSV file and inspect its shape, data types, first rows and the daily traffic trend (Step 1).

4. Clean the data – remove 40 duplicate log lines, repair the latency (linear interpolation per access point) and packet-loss (per-access-point median) gaps and clip negative values to zero (Step 2).

5. Engineer features – upload/download ratio, megabytes per device and four ratios that compare the current hour with the median of the same hour of the previous weeks (12 features in all) – and analyse hourly usage, incident signatures and correlations (Step 3).

6. Split the data chronologically (training: 2025-01-15 to 2025-03-18; validation: 2025-03-19 to 2025-04-08; testing: 2025-04-09 to 2025-04-30) and tune the Isolation Forest with a grid search scored by F1 on the validation window (Step 4).

7. Train the final IsolationForest with the best parameters (100 trees, max_samples 1024, alert rate 3%), plot the quality against the number of trees and save the model to models/wifi_iforest_model.joblib (Step 4).

8. Score all hours of a quiet day and of an incident-heavy day with the trained model and obtain the alerts with their severity and probable cause (Step 5).

9. Evaluate the model on the held-out test period – Precision, Recall, F1, ROC-AUC, PR-AUC, comparison with the Z-score rule and Local Outlier Factor, detection by incident type and alert workload (Step 6).

10. Log every alert (alert time, access point, score, severity, probable cause, ground truth, timestamp) into the SQLite database database/alerts.db and display the alert history (Step 7).

Implementation Code:

Core modules of the project (identical to the source files in the project ZIP). The plotting module (src/visualize.py), the command-line runner (run_all.py) and the notebook are provided in the ZIP.

1. Synthetic Dataset Generation (data/generate_dataset.py)

```
"""Generate a reproducible SYNTHETIC dataset of hourly campus Wi-Fi access-point logs.

No real network logs are used. Every row is produced from the explicit rules below
with a seeded NumPy generator, so the CSV is fully reproducible. Four kinds of
incidents are injected and labelled ONLY for evaluation (the model never sees them).
Replace data/wifi_traffic.csv with real controller logs (same columns) to retrain.

Run:  python data/generate_dataset.py
"""
from pathlib import Path
import numpy as np
import pandas as pd

SEED = 42
START, END = "2025-01-01 00:00", "2025-04-30 23:00"
N_INCIDENTS = 260

# ap_id: (zone, base devices, MB per device-hour, upload share of download)
APS = {
    "LIB-1":  ("academic", 180, 55, 0.12), "LIB-2":  ("academic", 140, 55, 0.12),
    "HST-A":  ("hostel",   320, 90, 0.10), "HST-B":  ("hostel",   280, 90, 0.10),
    "HST-C":  ("hostel",   240, 90, 0.10), "LAB-CSE": ("academic", 120, 70, 0.18),
    "LAB-ECE": ("academic", 90, 60, 0.16), "ADMIN":  ("office",    70, 35, 0.15),
    "CANTEEN": ("canteen", 150, 20, 0.08), "AUDI":   ("academic",  60, 60, 0.12),
}
INCIDENTS = ["Data exfiltration", "Brute-force login", "Traffic flood", "AP hardware fault"]
AUDI_EVENT_DAYS = ["2025-01-18", "2025-02-08", "2025-02-22", "2025-03-15", "2025-04-05"]


def zone_profile(zone, hour, weekend):
    """Relative load (0..1) of a zone for each hour of the day."""
    h = hour.astype(float)
    if zone == "academic":
        p = 0.05 + 0.95 * np.exp(-((h - 13) / 4.5) ** 2)
        return p * np.where(weekend, 0.30, 1.0)
    if zone == "office":
        p = 0.03 + 0.97 * np.exp(-((h - 12.5) / 3.2) ** 2)
        return p * np.where(weekend, 0.10, 1.0)
    if zone == "canteen":
        p = 0.05 + sum(np.exp(-((h - c) / 1.1) ** 2) for c in (8.5, 13.0, 17.0))
        return np.minimum(p, 1.0) * np.where(weekend, 0.60, 1.0)
    d = np.minimum(np.abs(h - 21), 24 - np.abs(h - 21))            # hostel: evening peak
    p = 0.30 + 0.70 * np.exp(-(d / 4.2) ** 2)
    return p * np.where(weekend, 1.15, 1.0)


def generate(start=START, end=END, seed=SEED):
    rng = np.random.default_rng(seed)
    times = pd.date_range(start, end, freq="h")
    n = len(times)
    hour, dow = times.hour.to_numpy(), times.dayofweek.to_numpy()
    weekend = dow >= 5
    frames = []
    for ap, (zone, base, mb_dev, up_share) in APS.items():
        load = zone_profile(zone, hour, weekend)
        if ap == "AUDI":                                    # legitimate events (not anomalies)
            day = times.strftime("%Y-%m-%d")
            ev = np.isin(day, AUDI_EVENT_DAYS) & (hour >= 10) & (hour <= 15)
            load = np.where(ev, 4.0, load)
        devices = rng.poisson(np.maximum(base * load * rng.normal(1, 0.06, n), 1))
        download = devices * mb_dev * rng.lognormal(0, 0.12, n)
        upload = download * up_share * rng.lognormal(0, 0.12, n)
        logins = rng.poisson(0.2 + 0.004 * devices)
        latency = 15 + 0.03 * devices + rng.normal(0, 1.5, n)
        loss = rng.gamma(2.0, 0.15, n)
        label = np.zeros(n, dtype=int)
        kind = np.array([""] * n, dtype=object)
        frames.append(dict(ap=ap, devices=devices.astype(float), download=download,
                           upload=upload, logins=logins.astype(float), latency=latency,
                           loss=loss, label=label, kind=kind))

    # ---- inject incidents (1-4 consecutive hours on one access point) -----------------
    for _ in range(N_INCIDENTS):
        f = frames[rng.integers(len(frames))]
        kind = INCIDENTS[rng.integers(len(INCIDENTS))]
        s = rng.uniform(0.5, 1.0)                           # severity: some are subtle
        dur = rng.choice([1, 2, 3, 4], p=[0.30, 0.35, 0.20, 0.15])
        i = rng.integers(0, n - dur)
        sl = slice(i, i + dur)
        if kind == "Data exfiltration":
            f["upload"][sl] *= 1 + s * rng.uniform(7, 14)
        elif kind == "Brute-force login":
            f["logins"][sl] += rng.poisson(s * rng.uniform(50, 140), dur)
        elif kind == "Traffic flood":
```

```
            f["download"][sl] *= 1 + s * rng.uniform(4, 8)
            f["upload"][sl] *= 1 + 2 * s
            f["latency"][sl] *= 1 + s * rng.uniform(2, 4)
            f["loss"][sl] *= 1 + s * rng.uniform(5, 9)
        else:                                           # AP hardware fault
            drop = 1 - s * rng.uniform(0.60, 0.85)
            f["devices"][sl] = np.round(f["devices"][sl] * drop)
            f["download"][sl] *= drop
            f["upload"][sl] *= drop
            f["latency"][sl] *= 1 + s * rng.uniform(3, 6)
            f["loss"][sl] *= 1 + s * rng.uniform(10, 20)
        f["label"][sl] = 1
        f["kind"][sl] = kind

    parts = []
    for f in frames:
        parts.append(pd.DataFrame({
            "timestamp": times, "ap_id": f["ap"], "zone": APS[f["ap"]][0],
            "connected_devices": f["devices"].astype(int),
            "download_mb": f["download"].round(1), "upload_mb": f["upload"].round(1),
            "failed_logins": f["logins"].astype(int),
            "avg_latency_ms": f["latency"].round(1),
            "packet_loss_pct": f["loss"].round(2).clip(0, 100),
            "is_anomaly": f["label"], "incident_type": f["kind"]}))
    df = pd.concat(parts).sort_values(["timestamp", "ap_id"]).reset_index(drop=True)

    # ---- data-quality problems handled later in preprocessing ---------------------------
    df.loc[rng.random(len(df)) < 0.015, "avg_latency_ms"] = np.nan      # controller gaps
    df.loc[rng.random(len(df)) < 0.012, "packet_loss_pct"] = np.nan
    dup = df.sample(40, random_state=seed)                              # duplicated log lines
    return pd.concat([df, dup]).sort_values(["timestamp", "ap_id"]).reset_index(drop=True)


if __name__ == "__main__":
    out = Path(__file__).resolve().parent / "wifi_traffic.csv"
    data = generate()
    data.to_csv(out, index=False)
    print(f"Saved {len(data)} rows to {out}")
```

2. Data Cleaning and Feature Engineering (src/preprocessing.py)

```
"""Data cleaning, feature engineering and chronological train / validation / test split."""
from dataclasses import dataclass
import numpy as np
import pandas as pd

LABEL, KIND = "is_anomaly", "incident_type"          # used ONLY for tuning / evaluation
RAW_FEATURES = ["connected_devices", "download_mb", "upload_mb", "failed_logins",
                "avg_latency_ms", "packet_loss_pct"]
DERIVED_FEATURES = ["upload_download_ratio", "mb_per_device"]
CALENDAR_FEATURES = ["hour", "day_of_week", "is_weekend"]
HISTORY_FEATURES = ["dev_vs_hist", "dl_vs_hist", "ul_vs_hist", "lat_vs_hist"]
# Calendar columns are kept for analysis but NOT fed to the detector: the history ratios already
# encode "what is normal for this hour", and extra calendar dimensions dilute the random splits
# (chosen on the validation window, see report).
FEATURES = RAW_FEATURES + DERIVED_FEATURES + HISTORY_FEATURES


def load_data(path):
    """Read the CSV, parse timestamps and sort chronologically."""
    df = pd.read_csv(path, parse_dates=["timestamp"])
    df[KIND] = df[KIND].fillna("none")                # empty = no incident (evaluation label only)
    return df.sort_values(["timestamp", "ap_id"]).reset_index(drop=True)


def clean_data(df):
    """Remove duplicate log lines, repair controller gaps and validate numeric ranges."""
    df = df.drop_duplicates(["timestamp", "ap_id"]).sort_values(["ap_id", "timestamp"]).copy()
    g = df.groupby("ap_id")
    df["avg_latency_ms"] = g["avg_latency_ms"].transform(
        lambda s: s.interpolate(limit_direction="both"))              # smooth variable
    df["packet_loss_pct"] = g["packet_loss_pct"].transform(
        lambda s: s.fillna(s.median()))                               # bursty variable
    for col in RAW_FEATURES:
        df[col] = df[col].clip(lower=0)
    df["packet_loss_pct"] = df["packet_loss_pct"].clip(upper=100)
    return df.sort_values(["timestamp", "ap_id"]).reset_index(drop=True)


def add_calendar_features(df):
    df = df.copy()
    df["hour"] = df["timestamp"].dt.hour
    df["day_of_week"] = df["timestamp"].dt.dayofweek
    df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)
    return df


def add_derived_features(df):
    df = df.copy()
    df["upload_download_ratio"] = df["upload_mb"] / (df["download_mb"] + 1)
    df["mb_per_device"] = df["download_mb"] / (df["connected_devices"] + 1)
    return df


def add_history_features(df):
    """Compare each hour with the same hour-of-week on the same AP in the previous weeks
    (median of up to 4 earlier weeks, shifted, so no future leakage)."""
    df = df.sort_values(["ap_id", "day_of_week", "hour", "timestamp"]).copy()
    grp = df.groupby(["ap_id", "day_of_week", "hour"])
    for col, name in [("connected_devices", "dev_vs_hist"), ("download_mb", "dl_vs_hist"),
                      ("upload_mb", "ul_vs_hist"), ("avg_latency_ms", "lat_vs_hist")]:
        hist = grp[col].transform(lambda s: s.shift(1).rolling(4, min_periods=2).median())
        df[name] = (df[col] + 1) / (hist + 1)
    return df.sort_values(["timestamp", "ap_id"]).reset_index(drop=True)


def build_features(raw_df):
    """Return X (features), y (1 = injected incident) and the engineered DataFrame."""
    df = add_history_features(add_derived_features(add_calendar_features(clean_data(raw_df))))
    df = df.dropna(subset=HISTORY_FEATURES).reset_index(drop=True)    # first 2 weeks per AP
    return df[FEATURES], df[LABEL], df


@dataclass
class Split:
    X_train: pd.DataFrame
    X_val: pd.DataFrame
    X_test: pd.DataFrame
    y_train: pd.Series
    y_val: pd.Series
    y_test: pd.Series
    df_train: pd.DataFrame
    df_val: pd.DataFrame
    df_test: pd.DataFrame
    cutoffs: tuple
```

```
def chronological_split(X, y, df, val_fraction=0.2, test_fraction=0.2):
    """Train on the earliest days, validate on the next, test on the most recent (no shuffling)."""
    days = np.sort(df["timestamp"].dt.normalize().unique())
    c_val = pd.Timestamp(days[int(len(days) * (1 - val_fraction - test_fraction))])
    c_test = pd.Timestamp(days[int(len(days) * (1 - test_fraction))])
    t = df["timestamp"]
    tr = (t < c_val).to_numpy()
    va = ((t >= c_val) & (t < c_test)).to_numpy()
    te = (t >= c_test).to_numpy()
    return Split(X[tr], X[va], X[te], y[tr], y[va], y[te], df[tr], df[va], df[te], (c_val, c_test))
```

3. Model Tuning and Training (src/train.py)

```
"""Tune, train and save the Isolation Forest anomaly detector."""
import itertools
import json
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.metrics import (average_precision_score, f1_score, precision_score,
                             recall_score, roc_auc_score)

RANDOM_STATE = 42
PARAM_GRID = {
    "n_estimators": [100, 200],
    "max_samples": [256, 1024],
    "max_features": [1.0, 0.6],
}
CONTAMINATIONS = [0.01, 0.02, 0.03, 0.05]


def anomaly_score(model, X):
    """Higher score = more abnormal (negated score_samples of the Isolation Forest)."""
    return -model.score_samples(X)


def threshold_from_train(train_scores, contamination):
    """Alert threshold = the (1 - contamination) quantile of the training scores."""
    return float(np.quantile(train_scores, 1 - contamination))


def tune_model(split):
    """Grid search. Each model is fitted on UNLABELLED training data only; the labelled
    validation window (a later period) is used to choose the settings by F1 score."""
    keys = list(PARAM_GRID)
    rows = []
    for values in itertools.product(*PARAM_GRID.values()):
        params = dict(zip(keys, values))
        model = IsolationForest(random_state=RANDOM_STATE, n_jobs=-1, **params).fit(split.X_train)
        s_tr, s_val = anomaly_score(model, split.X_train), anomaly_score(model, split.X_val)
        auc = roc_auc_score(split.y_val, s_val)
        for c in CONTAMINATIONS:
            pred = (s_val > threshold_from_train(s_tr, c)).astype(int)
            rows.append({**params, "contamination": c,
                         "val_precision": precision_score(split.y_val, pred),
                         "val_recall": recall_score(split.y_val, pred),
                         "val_f1": f1_score(split.y_val, pred), "val_auc": auc})
    return pd.DataFrame(rows).sort_values("val_f1", ascending=False).reset_index(drop=True)


def cv_results_table(results, top=5):
    return results.head(top).reset_index(drop=True)


def clean_params(best_params):
    """Keep only model arguments and restore integer types lost when a DataFrame row is read."""
    p = {k: best_params[k] for k in ("n_estimators", "max_samples", "max_features")}
    p["n_estimators"], p["max_samples"] = int(p["n_estimators"]), int(p["max_samples"])
    p["max_features"] = float(p["max_features"])
    return p


def fit_final_model(X_train, best_params):
    params = clean_params(best_params)
    return IsolationForest(random_state=RANDOM_STATE, n_jobs=-1, **params).fit(X_train)


def training_curve(split, best_params, max_trees=300, step=20):
    """Diagnostic: train / validation ROC-AUC and PR-AUC as trees are added."""
    params = {k: v for k, v in clean_params(best_params).items() if k != "n_estimators"}
    iso = IsolationForest(n_estimators=step, warm_start=True, random_state=RANDOM_STATE,
                          n_jobs=-1, **params)
    log = []
    for n_trees in range(step, max_trees + 1, step):
        iso.set_params(n_estimators=n_trees).fit(split.X_train)
        s_tr, s_val = anomaly_score(iso, split.X_train), anomaly_score(iso, split.X_val)
        log.append({"n_trees": n_trees,
                    "train_auc": roc_auc_score(split.y_train, s_tr),
                    "val_auc": roc_auc_score(split.y_val, s_val),
                    "val_pr_auc": average_precision_score(split.y_val, s_val)})
    return pd.DataFrame(log)


def save_model(model, path, best_params, threshold, cutoffs):
    path = Path(path)
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, path)
    meta = {"algorithm": "IsolationForest", "best_params": best_params,
            "alert_threshold": threshold,
            "train_val_test_cutoffs": [str(c.date()) for c in cutoffs],
            "features": list(model.feature_names_in_)}
    path.with_suffix(".json").write_text(json.dumps(meta, indent=2, default=str))
```

4. Model Evaluation (src/evaluate.py)

```
"""Detection metrics, baseline comparison and alert-workload analysis."""
import numpy as np
import pandas as pd
from sklearn.metrics import (average_precision_score, confusion_matrix, f1_score,
                             precision_score, recall_score, roc_auc_score)
from sklearn.neighbors import LocalOutlierFactor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import RobustScaler
from sklearn.svm import OneClassSVM

from .preprocessing import RAW_FEATURES
from .train import threshold_from_train


def detection_metrics(y_true, scores, pred):
    """Precision / Recall / F1 at the alert threshold, ROC-AUC and PR-AUC over all thresholds."""
    return {"Precision": precision_score(y_true, pred), "Recall": recall_score(y_true, pred),
            "F1": f1_score(y_true, pred), "ROC_AUC": roc_auc_score(y_true, scores),
            "PR_AUC": average_precision_score(y_true, scores), "Alerts": int(np.sum(pred))}


def baseline_scores(split, seed=42):
    """Two reference detectors the Isolation Forest must beat (higher score = more abnormal)."""
    out = {}
    med = split.X_train[RAW_FEATURES].median()
    mad = (split.X_train[RAW_FEATURES] - med).abs().median() * 1.4826 + 1e-9
    z = lambda X: ((X[RAW_FEATURES] - med) / mad).abs().max(axis=1).to_numpy()
    out["Z-score rule"] = (z(split.X_train), z(split.X_test))
    sub = split.X_train.sample(4000, random_state=seed)
    lof = make_pipeline(RobustScaler(), LocalOutlierFactor(n_neighbors=35, novelty=True)).fit(sub)
    out["Local Outlier Factor"] = (-lof.score_samples(split.X_train),
                                   -lof.score_samples(split.X_test))
    return out


def compare_models(split, if_scores, contamination):
    """Evaluate every detector on the unseen test period with the same alert-rate rule."""
    all_scores = {"Isolation Forest": if_scores, **baseline_scores(split)}
    rows, preds, scores = {}, {}, {}
    for name, (s_tr, s_te) in all_scores.items():
        pred = (s_te > threshold_from_train(s_tr, contamination)).astype(int)
        rows[name] = detection_metrics(split.y_test, s_te, pred)
        preds[name], scores[name] = pred, s_te
    return pd.DataFrame(rows).T.round(3), preds, scores


def recall_by_type(df_test, pred):
    """Share of each injected incident type that raised at least one hourly alert."""
    d = df_test.assign(pred=pred)
    d = d[d["is_anomaly"] == 1]
    return d.groupby("incident_type")["pred"].agg(incidents_hours="size", detected="sum",
                                                  recall="mean").round(3)


def alert_workload(split, preds):
    """Alerts the network team would receive per day, and how many were false alarms."""
    days = split.df_test["timestamp"].dt.normalize().nunique()
    rows = {}
    for name, pred in preds.items():
        tn, fp, fn, tp = confusion_matrix(split.y_test, pred).ravel()
        rows[name] = {"alerts_per_day": (tp + fp) / days, "false_alarms_per_day": fp / days,
                      "missed_incident_hours": fn, "caught_incident_hours": tp}
    return pd.DataFrame(rows).T.round(2)
```

5. Alert Generation and Saving Alert History (src/detect.py)

```
"""Load the trained model, raise alerts for a day of traffic and log them to SQLite."""
import datetime as dt
import sqlite3
from pathlib import Path
import joblib
import pandas as pd

from .train import anomaly_score


def load_model(path="models/wifi_iforest_model.joblib"):
    return joblib.load(path)


def probable_cause(row):
    """Simple, explainable rules that turn an alert into a likely cause for the operator."""
    if row["failed_logins"] >= 12:
        return "Brute-force login"
    if row["upload_download_ratio"] >= 0.45:
        return "Data exfiltration"
    if row["dev_vs_hist"] < 0.5 and row["lat_vs_hist"] > 1.8:
        return "AP hardware fault"
    if row["dl_vs_hist"] > 2.5 and row["lat_vs_hist"] > 1.8:
        return "Traffic flood"
    return "Unclassified"


def severity(margin):
    """Severity from how far the score is above the alert threshold."""
    return "Critical" if margin > 0.10 else "High" if margin > 0.05 else "Medium"


def detect_day(model, X, df, date, threshold):
    """Return one row per alert raised on `date` (all access points, all 24 hours)."""
    mask = (df["timestamp"].dt.normalize() == pd.Timestamp(date)).to_numpy()
    out = df.loc[mask, ["timestamp", "ap_id", "connected_devices", "download_mb", "upload_mb",
                        "failed_logins", "is_anomaly", "incident_type"]].copy()
    out["anomaly_score"] = anomaly_score(model, X[mask]).round(4)
    out["probable_cause"] = [probable_cause(r) for _, r in df.loc[mask].iterrows()]
    out["severity"] = [severity(s - threshold) for s in out["anomaly_score"]]
    out = out[out["anomaly_score"] > threshold]
    out = out.rename(columns={"is_anomaly": "true_incident", "incident_type": "true_type"})
    return out.reset_index(drop=True)


def log_alerts(alert_df, db_path="database/alerts.db"):
    """Persist alerts (time, access point, score, severity, cause, ground truth, timestamp)."""
    Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(db_path)
    conn.execute("""CREATE TABLE IF NOT EXISTS alert_log (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        alert_time TEXT, ap_id TEXT, anomaly_score REAL, severity TEXT,
                        probable_cause TEXT, true_incident INTEGER, created_at TEXT)""")
    now = dt.datetime.now().isoformat(timespec="seconds")
    conn.executemany(
        "INSERT INTO alert_log (alert_time, ap_id, anomaly_score, severity, probable_cause,"
        " true_incident, created_at) VALUES (?, ?, ?, ?, ?, ?, ?)",
        [(str(r.timestamp), r.ap_id, float(r.anomaly_score), r.severity, r.probable_cause,
          int(r.true_incident), now) for r in alert_df.itertuples()])
    conn.commit()
    conn.close()


def read_history(db_path="database/alerts.db", limit=10):
    conn = sqlite3.connect(db_path)
    hist = pd.read_sql_query("SELECT * FROM alert_log ORDER BY id DESC LIMIT ?",
                             conn, params=(limit,))
    conn.close()
    return hist
```

Output (Step-by-Step Screenshots):

All outputs below were produced by executing the notebook's code cells. JupyterLab could not be installed in the environment used to generate this report, so the cells were run with a scripted executor (tools/build_notebook_and_screenshots.py) and each figure is a rendering of the actual code and captured output of that cell – it is not a screenshot of the JupyterLab window. No value has been edited by hand.

Smart Campus Wi-Fi Anomaly Detection — Step 1 · Load dataset — Python 3 · Isolation Forest

In [2]:

```
CSV = ROOT / "data" / "wifi_traffic.csv"
if not CSV.exists():
    generate().to_csv(CSV, index=False)        # synthetic, seeded (seed = 42)
raw = load_data(CSV)
print("Shape:", raw.shape, "| Period:", raw["timestamp"].min(), "to", raw["timestamp"].max())
print("Access points:", sorted(raw["ap_id"].unique()), "\n")
display(raw.head(8))
raw.info()
```

```
Shape: (28840, 11) | Period: 2025-01-01 00:00:00 to 2025-04-30 23:00:00
Access points: ['ADMIN', 'AUDI', 'CANTEEN', 'HST-A', 'HST-B', 'HST-C', 'LAB-CSE', 'LAB-ECE', 'LIB-1', 'LIB-2']
```

	timestamp	ap_id	zone	connected_devices	download_mb	upload_mb	failed_logins	avg_latency_ms	packet_loss_pct	is_anomaly	incident_type
0	2025-01-01	ADMIN	office	4	138.9	18.9	0	13.1	0.34	0	none
1	2025-01-01	AUDI	academic	2	123.2	15.1	0	17.0	0.16	0	none
2	2025-01-01	CANTEEN	canteen	7	139.8	10.0	1	13.1	0.03	0	none
3	2025-01-01	HST-A	hostel	235	19617.7	2008.2	0	22.6	0.02	0	none
4	2025-01-01	HST-B	hostel	170	16385.0	2202.4	0	22.2	0.14	0	none
5	2025-01-01	HST-C	hostel	185	14083.0	1425.4	0	19.7	0.60	0	none
6	2025-01-01	LAB-CSE	academic	6	393.3	72.3	0	12.6	0.64	0	none
7	2025-01-01	LAB-ECE	academic	2	132.3	18.0	98	14.0	0.15	1	Brute-force login

```
<class 'pandas.DataFrame'>
RangeIndex: 28840 entries, 0 to 28839
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   timestamp          28840 non-null  datetime64[us]
 1   ap_id              28840 non-null  str
 2   zone               28840 non-null  str
 3   connected_devices  28840 non-null  int64
 4   download_mb        28840 non-null  float64
 5   upload_mb          28840 non-null  float64
 6   failed_logins      28840 non-null  int64
 7   avg_latency_ms     28410 non-null  float64
 8   packet_loss_pct    28494 non-null  float64
 9   is_anomaly         28840 non-null  int64
 10  incident_type      28840 non-null  str
dtypes: datetime64[us](1), float64(4), int64(3), str(3)
memory usage: 2.4 MB
```

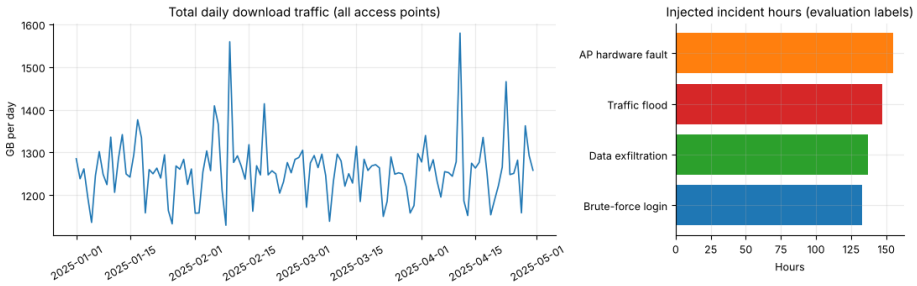
Rendered from the executed notebook cell (code and captured output).

Fig 2: Step 1 — Load dataset: shape, date range, first rows and column information of the Wi-Fi log data (28,840 records, 11 columns)

Smart Campus Wi-Fi Anomaly Detection — Step 1 · Traffic overview — Python 3 · Isolation Forest

In [3]:

```
fig = viz.plot_daily_traffic(raw); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 3: Step 1 — Total daily download traffic and the number of injected incident hours of each type

Smart Campus Wi-Fi Anomaly Detection — Step 2 · Preprocessing — Python 3 · Isolation Forest

In [4]:
```
print("Missing values BEFORE cleaning:")
print(raw.isna().sum()[lambda s: s > 0].to_string())
print("Duplicate (timestamp, ap_id) log lines:", int(raw.duplicated(["timestamp", "ap_id"]).sum()))
clean = clean_data(raw)
print("\nMissing values AFTER cleaning:", int(clean.isna().sum().sum()), "| rows:", len(clean))
X, y, df = build_features(raw)
print(f"Rows after history features: {len(df)} (first 2 weeks of each AP dropped) | Features: {X.shape[1]}")
display(X.head(3).T)
```

```
Missing values BEFORE cleaning:
avg_latency_ms     430
packet_loss_pct    346
Duplicate (timestamp, ap_id) log lines: 40

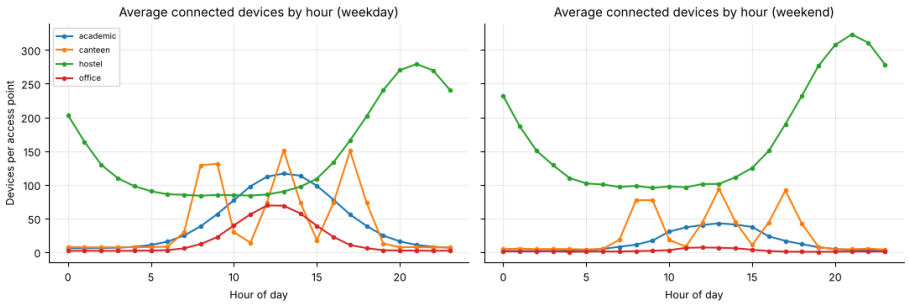
Missing values AFTER cleaning: 0 | rows: 28800
Rows after history features: 25440 (first 2 weeks of each AP dropped) | Features: 12
                              0            1           2
connected_devices         4.000000     4.000000   15.000000
download_mb             143.000000  1204.800000  278.700000
upload_mb                23.600000    53.600000   22.800000
failed_logins             1.000000     0.000000    1.000000
avg_latency_ms           17.300000    43.700000   17.600000
packet_loss_pct           0.330000     1.410000    0.410000
upload_download_ratio     0.163889     0.044452    0.081516
mb_per_device            28.600000   240.960000   17.418750
dev_vs_hist               1.250000     1.111111    1.882353
dl_vs_hist                1.434978     5.408388    1.774746
ul_vs_hist                1.587097     2.192771    1.888889
lat_vs_hist               1.165605     2.725610    1.223684
```

Rendered from the executed notebook cell (code and captured output).

Fig 4: Step 2 — Preprocessing: missing values before and after cleaning, duplicates removed and the 12-feature matrix (first three rows, transposed)

Smart Campus Wi-Fi Anomaly Detection — Step 3 · Hourly usage patterns — Python 3 · Isolation Forest

In [5]:
```
fig = viz.plot_hourly_patterns(df); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 5: Step 3 — Exploratory analysis: average connected devices by hour for each zone on weekdays and weekends

Smart Campus Wi-Fi Anomaly Detection — Step 3 · Incident signatures — Python 3 · Isolation Forest

In [6]:
```
fig = viz.plot_incident_signatures(df); plt.show()
```

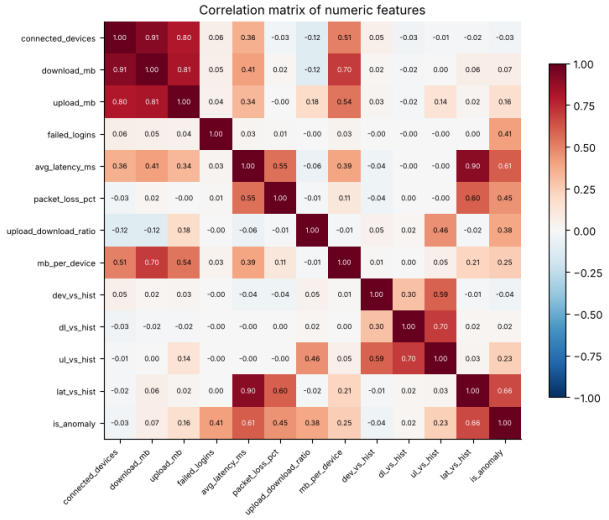


Rendered from the executed notebook cell (code and captured output).

Fig 6: Step 3 — Signature of each incident type compared with normal hours

Smart Campus Wi-Fi Anomaly Detection — Step 3 · Feature correlation — Python 3 · Isolation Forest

In [7]:
```
fig = viz.plot_correlation(df, FEATURES + ["is_anomaly"]); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 7: Step 3 — Correlation matrix of the features and the incident label

Smart Campus Wi-Fi Anomaly Detection — Step 4 · Hyper-parameter tuning — Python 3 · Isolation Forest

In [8]:
```
split = chronological_split(X, y, df)
for name, Xs, d in [("Train", split.X_train, split.df_train), ("Valid", split.X_val, split.df_val),
                    ("Test ", split.X_test, split.df_test)]:
    print(f"{name}: {len(Xs)} rows ({d['timestamp'].min().date()} to {d['timestamp'].max().date()})")
results = tune_model(split)
best = results.iloc[0].to_dict()
print("\nBest parameters:", clean_params(best), "| contamination =", best["contamination"])
print("Best validation F1: %.3f | validation ROC-AUC: %.3f" % (best["val_f1"], best["val_auc"]))
display(cv_results_table(results).round(3))
```

```
Train: 15120 rows (2025-01-15 to 2025-03-18)
Valid: 5040 rows (2025-03-19 to 2025-04-08)
Test : 5280 rows (2025-04-09 to 2025-04-30)

Best parameters: {'n_estimators': 100, 'max_samples': 1024, 'max_features': 1.0} | contamination = 0.03
Best validation F1: 0.748 | validation ROC-AUC: 0.994
```

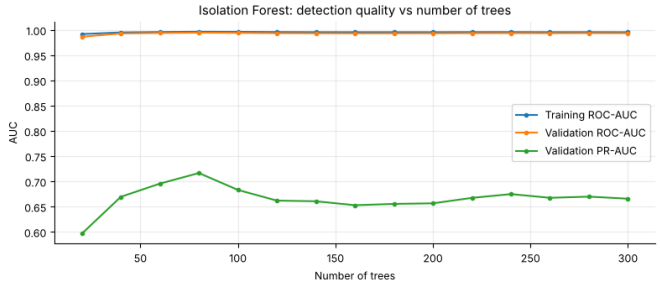
	n_estimators	max_samples	max_features	contamination	val_precision	val_recall	val_f1	val_auc
0	100	1024	1.0	0.03	0.608	0.973	0.748	0.994
1	100	1024	1.0	0.02	0.715	0.779	0.746	0.994
2	200	1024	1.0	0.02	0.709	0.735	0.722	0.993
3	200	1024	1.0	0.03	0.553	0.920	0.691	0.993
4	100	256	1.0	0.02	0.653	0.681	0.667	0.986

Rendered from the executed notebook cell (code and captured output).

Fig 8: Step 4 — Chronological train / validation / test split and Isolation Forest hyper-parameter search (validation F1)

Smart Campus Wi-Fi Anomaly Detection — Step 4 · Training curve — Python 3 · Isolation Forest

In [9]:
```
log = training_curve(split, best)
fig = viz.plot_training_curve(log); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

Fig 9: Step 4 — Isolation Forest detection quality versus number of trees

Smart Campus Wi-Fi Anomaly Detection — Step 4 · Training log — Python 3 · Isolation Forest

In [10]:
```
for r in log.itertuples():
    print(f"[Trees {r.n_trees:3d}/300]  train AUC = {r.train_auc:.4f} | "
          f"validation AUC = {r.val_auc:.4f} | validation PR-AUC = {r.val_pr_auc:.4f}")
model = fit_final_model(split.X_train, best)
s_train, s_test = anomaly_score(model, split.X_train), anomaly_score(model, split.X_test)
thr = threshold_from_train(s_train, best["contamination"])
save_model(model, ROOT / "models" / "wifi_iforest_model.joblib",
           {**clean_params(best), "contamination": best["contamination"]}, thr, split.cutoffs)
print(f"\nFinal model trained on {len(split.X_train)} rows -> models/wifi_iforest_model.joblib")
print(f"Alert threshold (anomaly score) = {thr:.4f}")
```

```
[Trees  20/300]  train AUC = 0.9920 | validation AUC = 0.9867 | validation PR-AUC = 0.5966
[Trees  40/300]  train AUC = 0.9953 | validation AUC = 0.9931 | validation PR-AUC = 0.6696
[Trees  60/300]  train AUC = 0.9960 | validation AUC = 0.9942 | validation PR-AUC = 0.6957
[Trees  80/300]  train AUC = 0.9966 | validation AUC = 0.9948 | validation PR-AUC = 0.7167
[Trees 100/300]  train AUC = 0.9964 | validation AUC = 0.9944 | validation PR-AUC = 0.6832
[Trees 120/300]  train AUC = 0.9960 | validation AUC = 0.9938 | validation PR-AUC = 0.6621
[Trees 140/300]  train AUC = 0.9958 | validation AUC = 0.9935 | validation PR-AUC = 0.6608
[Trees 160/300]  train AUC = 0.9957 | validation AUC = 0.9933 | validation PR-AUC = 0.6530
[Trees 180/300]  train AUC = 0.9957 | validation AUC = 0.9934 | validation PR-AUC = 0.6555
[Trees 200/300]  train AUC = 0.9958 | validation AUC = 0.9935 | validation PR-AUC = 0.6567
[Trees 220/300]  train AUC = 0.9959 | validation AUC = 0.9937 | validation PR-AUC = 0.6675
[Trees 240/300]  train AUC = 0.9959 | validation AUC = 0.9939 | validation PR-AUC = 0.6750
[Trees 260/300]  train AUC = 0.9958 | validation AUC = 0.9937 | validation PR-AUC = 0.6676
[Trees 280/300]  train AUC = 0.9958 | validation AUC = 0.9938 | validation PR-AUC = 0.6699
[Trees 300/300]  train AUC = 0.9957 | validation AUC = 0.9937 | validation PR-AUC = 0.6659

Final model trained on 15120 rows -> models/wifi_iforest_model.joblib
Alert threshold (anomaly score) = 0.4878
```

Rendered from the executed notebook cell (code and captured output).

Fig 10: Step 4 — Training log (15 checkpoints up to 300 trees) and final model saved to models/wifi_iforest_model.joblib

Smart Campus Wi-Fi Anomaly Detection — Step 5 · Detection: quiet day — Python 3 · Isolation Forest

In [11]:
```
test = split.df_test
def show_day(date):
    alerts = detect_day(model, split.X_test, test, date, thr)
    day = test[test["timestamp"].dt.normalize() == pd.Timestamp(date)]
    print("Scenario:", pd.Timestamp(date).strftime("%A, %d %b %Y"),
          f"| records: {len(day)} | true incident hours: {int(day['is_anomaly'].sum())} | alerts: {len(alerts)}\n")
    display(alerts[["timestamp", "ap_id", "anomaly_score", "severity", "probable_cause",
                    "true_incident", "true_type"]].head(8))
    return alerts

per_day = test.groupby(test["timestamp"].dt.normalize())["is_anomaly"].sum()
d1 = per_day.idxmin()                          # quietest day (fewest real incident hours)
_ = show_day(d1)
```

```
Scenario: Monday, 14 Apr 2025 | records: 240 | true incident hours: 1 | alerts: 2

             timestamp  ap_id  anomaly_score severity     probable_cause  true_incident          true_type
0 2025-04-14 10:00:00   AUDI         0.4981   Medium  Brute-force login              1  Brute-force login
1 2025-04-14 21:00:00  HST-A         0.5076   Medium       Unclassified              0               none
```

Rendered from the executed notebook cell (code and captured output).

Fig 11: Step 5 — Alerts for the quietest day of the test period

Smart Campus Wi-Fi Anomaly Detection — Step 5 · Detection: incident day — Python 3 · Isolation Forest

In [12]:
```
d2 = per_day.idxmax()                          # day with the most incident hours
_ = show_day(d2)
```

```
Scenario: Wednesday, 30 Apr 2025 | records: 240 | true incident hours: 14 | alerts: 14

             timestamp    ap_id  anomaly_score  severity     probable_cause  true_incident          true_type
0 2025-04-30 03:00:00    HST-C         0.6357  Critical  Data exfiltration              1  Data exfiltration
1 2025-04-30 04:00:00    HST-C         0.5630      High  Data exfiltration              1  Data exfiltration
2 2025-04-30 04:00:00  LAB-CSE         0.6006  Critical  Data exfiltration              1  Data exfiltration
3 2025-04-30 05:00:00  LAB-CSE         0.6504  Critical  Data exfiltration              1  Data exfiltration
4 2025-04-30 06:00:00  LAB-CSE         0.6429  Critical  Data exfiltration              1  Data exfiltration
5 2025-04-30 11:00:00  LAB-CSE         0.6915  Critical  AP hardware fault              1  AP hardware fault
6 2025-04-30 12:00:00    HST-B         0.5062    Medium  Brute-force login              1  Brute-force login
7 2025-04-30 13:00:00    HST-B         0.5136    Medium  Brute-force login              1  Brute-force login
```

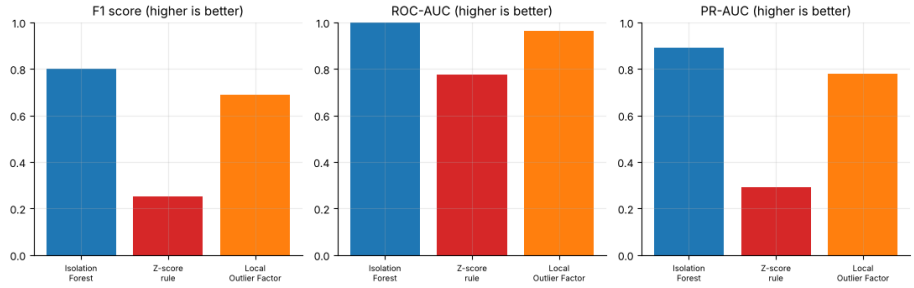
Rendered from the executed notebook cell (code and captured output).

Fig 12: Step 5 — Alerts for the day with the most incident hours, with severity and probable cause

Smart Campus Wi-Fi Anomaly Detection — Step 6 · Performance metrics & baselines — Python 3 · Isolation Forest

In [13]:
```
pred = (s_test > thr).astype(int)
m = detection_metrics(split.y_test, s_test, pred)
print("Isolation Forest - hold-out test metrics")
for k in ["Precision", "Recall", "F1", "ROC_AUC", "PR_AUC"]:
    print(f"  {k:9s}: {m[k]:.4f}")
table, preds, scores = compare_models(split, (s_train, s_test), best["contamination"])
display(table)
fig = viz.plot_model_comparison(table); plt.show()
```

```
Isolation Forest - hold-out test metrics
  Precision: 0.6836
  Recall   : 0.9680
  F1       : 0.8013
  ROC_AUC  : 0.9976
  PR_AUC   : 0.8899
                      Precision  Recall     F1  ROC_AUC  PR_AUC  Alerts
Isolation Forest            0.684   0.968  0.801    0.998   0.890   177.0
Z-score rule                0.234   0.272  0.252    0.777   0.291   145.0
Local Outlier Factor        0.601   0.808  0.689    0.964   0.779   168.0
```

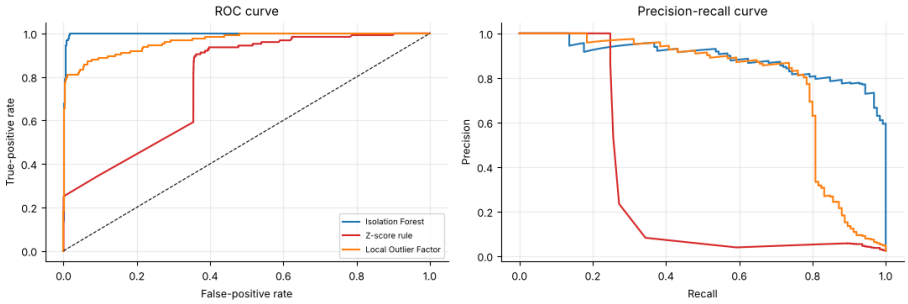


Rendered from the executed notebook cell (code and captured output).

Fig 13: Step 6 — Performance metrics (Precision 0.684, Recall 0.968, F1 0.801, ROC-AUC 0.998) and comparison with baseline detectors

Smart Campus Wi-Fi Anomaly Detection — Step 6 · ROC and precision-recall curves — Python 3 · Isolation Forest

In [14]:
```
fig = viz.plot_roc_pr(split.y_test, scores); plt.show()
```

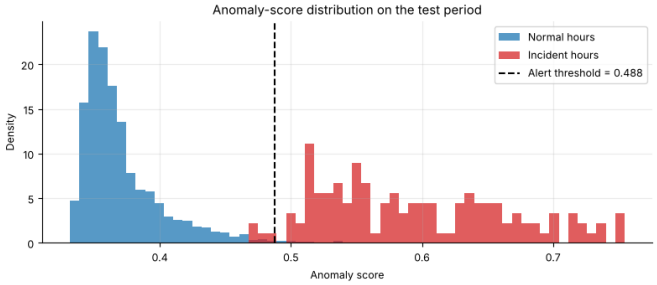


Rendered from the executed notebook cell (code and captured output).

Fig 14: Step 6 — ROC and precision-recall curves of the three detectors

Smart Campus Wi-Fi Anomaly Detection — Step 6 · Anomaly-score distribution — Python 3 · Isolation Forest

In [15]:
```
fig = viz.plot_score_distribution(s_test, split.y_test.to_numpy(), thr); plt.show()
```

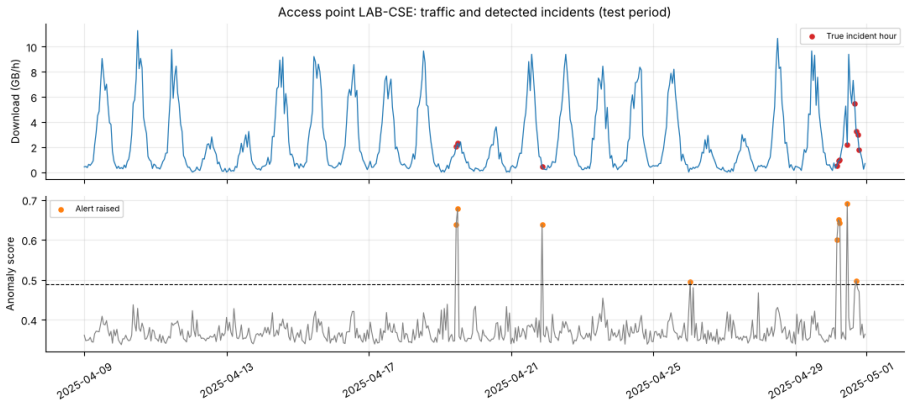


Rendered from the executed notebook cell (code and captured output).

Fig 15: Step 6 — Anomaly-score distribution of normal and incident hours with the alert threshold

Smart Campus Wi-Fi Anomaly Detection — Step 6 · Detected incidents on one access point — Python 3 · Isolation Forest

In [16]:
```
fig = viz.plot_timeline(split.df_test, s_test, thr, "LAB-CSE"); plt.show()
```



Rendered from the executed notebook cell (code and captured output).

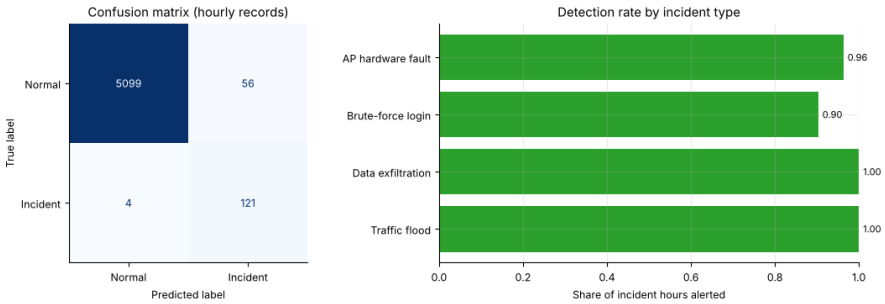
Fig 16: Step 6 — Traffic, true incident hours and alerts on one access point during the test period

Smart Campus Wi-Fi Anomaly Detection — Step 6 · Alert workload & detection by incident type — Python 3 · Isolation Forest

In [17]:
```
by_type = recall_by_type(split.df_test, preds["Isolation Forest"])
display(by_type)
display(alert_workload(split, preds))
fig = viz.plot_confusion_and_types(split.y_test, preds["Isolation Forest"], by_type); plt.show()
```

incident_type	incidents_hours	detected	recall
AP hardware fault	27	26	0.963
Brute-force login	31	28	0.903
Data exfiltration	35	35	1.000
Traffic flood	32	32	1.000

	alerts_per_day	false_alarms_per_day	missed_incident_hours	caught_incident_hours
Isolation Forest	8.05	2.55	4.0	121.0
Z-score rule	6.59	5.05	91.0	34.0
Local Outlier Factor	7.64	3.05	24.0	101.0



Rendered from the executed notebook cell (code and captured output).

Fig 17: Step 6 — Detection rate by incident type, alert workload and confusion matrix

Smart Campus Wi-Fi Anomaly Detection — Step 7 · Alert history (SQLite) — Python 3 · Isolation Forest

In [18]:
```
db = ROOT / "database" / "alerts.db"
db.unlink(missing_ok=True)                    # start with a fresh log
for d in (d1, d2):
    log_alerts(detect_day(model, split.X_test, test, d, thr), db)
display(read_history(db, limit=10))
```

	id	alert_time	ap_id	anomaly_score	severity	probable_cause	true_incident	created_at
0	16	2025-04-30 22:00:00	HST-B	0.5145	Medium	Unclassified	0	2026-10-16T03:38:00
1	15	2025-04-30 21:00:00	HST-A	0.5173	Medium	Unclassified	0	2026-10-16T03:38:00
2	14	2025-04-30 20:00:00	HST-A	0.4945	Medium	Unclassified	0	2026-10-16T03:38:00
3	13	2025-04-30 17:00:00	LAB-CSE	0.4977	Medium	Brute-force login	1	2026-10-16T03:38:00
4	12	2025-04-30 15:00:00	HST-B	0.5142	Medium	Brute-force login	1	2026-10-16T03:38:00
5	11	2025-04-30 14:00:00	HST-B	0.5148	Medium	Brute-force login	1	2026-10-16T03:38:00
6	10	2025-04-30 13:00:00	HST-B	0.5136	Medium	Brute-force login	1	2026-10-16T03:38:00
7	9	2025-04-30 12:00:00	HST-B	0.5062	Medium	Brute-force login	1	2026-10-16T03:38:00
8	8	2025-04-30 11:00:00	LAB-CSE	0.6915	Critical	AP hardware fault	1	2026-10-16T03:38:00
9	7	2025-04-30 06:00:00	LAB-CSE	0.6429	Critical	Data exfiltration	1	2026-10-16T03:38:00

Rendered from the executed notebook cell (code and captured output).

Fig 18: Step 7 — Alert history persisted in the SQLite database

Result:

A Smart Campus Wi-Fi Anomaly Detection system was successfully designed and implemented in Python 3 using pandas, scikit-learn (Isolation Forest) and an SQLite database, as a JupyterLab notebook with a command-line runner. After cleaning and feature engineering (12 features), the tuned model (100 trees, max_samples 1024, alert rate 3%) was fitted without labels on the earliest 15,120 records, tuned on the next 5,040 and tested on the latest 5,280 records (2025-04-09 to 2025-04-30). On this held-out test period it achieved Precision 0.684, Recall 0.968, F1 0.801, ROC-AUC 0.998 and PR-AUC 0.890, compared with an F1 of 0.252 for the Z-score rule and 0.689 for Local Outlier Factor. It caught 121 of 125 incident hours and raised 8.1 alerts per day, of which 2.5 were false alarms. Detection rates were 0.96 for ap hardware fault, 0.90 for brute-force login, 1.00 for data exfiltration, 1.00 for traffic flood. Every alert is logged to an alert-history database.

These figures must be read with care. First, the dataset is synthetic and its incidents are deliberately large compared with normal variation, so the results show that the pipeline works and recovers the simulated patterns, not how accurate it would be on a real campus. Second, the false alarms are not evenly spread: 32 of the 56 false-alarm hours came from HST-A, 31 of them between 19:00 and 23:00, when the hostel load and its natural hour-to-hour swings are largest. Third, the severity label is informative: the share of alerts that were real incidents was 92% for Critical, 74% for High and 43% for Medium alerts, so Medium alerts should be reviewed in batches. Fourth, the rule-based probable cause was right for 90% of the correctly alerted hours; the remainder were left as Unclassified rather than guessed. Finally, the choice to leave the calendar columns out of the detector was made by comparing validation scores, not test scores, and the alert rate was chosen on the validation window using the incident labels.

In conclusion, an Isolation Forest given per-hour behaviour features detects the injected incidents far more reliably than a fixed statistical rule on the simulated data, needs no labelled incidents for training, and produces alerts that can be ranked by severity and logged. The next steps are to retrain on real access-point controller logs, to review the hostel evening false alarms with per-zone thresholds, and to add streaming scoring so that alerts are raised within the hour in which the incident starts.